

東北大學



NORTHEASTERN
UNIVERSITY

毕业设计（论文）

GRADUATION DESIGN (THESIS)

论文题目

东北大学论文编译器设计与实现

学院名称 软件学院

专业名称 软件工程

学生姓名 Claude

指导教师 modulus

二〇二五年 六月

学号 2025DEMO

密级

东北大学本科毕业论文

东北大学论文编译器设计与实现

学院名称: 软件学院

专业名称: 软件工程

学生姓名: Claude

指导教师: modulus

2025年6月

Design and Implementation of NEU Thesis Compiler

by Claude

Supervisor: Professor modulus
Associate Supervisor: Professor

Northeastern University

June 2025

郑重声明

本人呈交的学位论文，是在导师的指导下，独立进行研究工作所取得的成果，所有数据、图片资料真实可靠。尽我所知，除文中已经注明引用的内容外，本学位论文的研究成果不包含他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确的方式标明。本学位论文的知识产权归属于培养单位。

本人签名：_____ 日期：_____

摘要

东北大学本科毕业设计（论文）的格式规范严格、排版要求细致，传统手工排版方式效率低下且易出错。本文设计并实现了一套基于扩展 Markdown DSL 的论文编译器系统，将论文源文件自动转换为符合东北大学书写印制规范的 Word 文档。系统采用三阶段流水线架构：Markdown 词法分析、AST 构建和.docx 文档生成，通过声明式的 DSL 语法支持图片引用、PlantUML 图、代码块和表格等学术排版元素，并内置内容检查器对摘要字数、关键词数量、章节比例、参考文献引用完整性等规范要求进行自动校验。

本文首先分析了东北大学论文书写印制规范的具体要求，包括页边距、字体字号、行间距、图表编号等数十项排版细则。在此基础上，设计了面向学术论文的扩展 Markdown DSL 语法，使作者能够以纯文本方式编写论文内容。系统的词法分析器将源文件解析为 Token 流，语法分析器构建 Thesis 抽象语法树，文档生成器根据 AST 和样式配置自动生成格式合规的 Word 文档。内容检查器对 AST 执行十余项规范校验，涵盖元数据完整性、摘要字数范围、章节结构完整性、各章篇幅比例和参考文献引用顺序等检查项。实验结果表明，该系统能够正确编译 DSL 源文件并生成符合格式要求的 Word 文档，有效提升了论文排版的效率和质量。

关键词：论文编译器；Markdown；文档生成；AST；python-docx

ABSTRACT

This paper designs and implements a thesis compiler system based on extended Markdown DSL, which automatically converts thesis source files into Word documents compliant with Northeastern University formatting specifications. The system adopts a three-stage pipeline architecture: Markdown tokenization, AST construction, and .docx generation. Through a declarative DSL syntax, it supports academic formatting elements such as figure references, PlantUML diagrams, code blocks, and tables. A built-in content checker automatically validates compliance requirements including abstract length, keyword count, chapter ratios, and reference citation integrity.

The paper first analyzes the specific requirements of the NEU thesis formatting specification, which includes dozens of typesetting rules covering margins, fonts, line spacing, and figure numbering. Based on this analysis, an extended Markdown DSL syntax is designed for academic writing. The tokenizer parses source files into token streams, the parser constructs a Thesis AST, and the document builder generates properly formatted Word documents from the AST and style configuration. Experimental results demonstrate that the system correctly compiles DSL source files and produces format-compliant Word documents, effectively improving thesis formatting efficiency and quality.

Key words: Thesis Compiler; Markdown; Document Generation; AST; python-docx

目 录

1 绪论.....	1
1.1 研究背景.....	1
1.2 系统目标.....	1
1.3 论文组织结构.....	1
2 相关技术.....	3
2.1 python-docx.....	3
2.2 PyYAML.....	3
2.3 PlantUML.....	3
2.4 本章小结.....	3
3 需求分析.....	5
3.1 功能需求.....	5
3.1.1 DSL 语法支持.....	5
3.1.2 文档生成.....	5
3.1.3 内容检查.....	5
3.2 非功能需求.....	5
3.3 用例分析.....	5
3.4 本章小结.....	6
4 系统设计.....	7
4.1 总体架构.....	7

4.2 数据模型设计.....8

4.3 词法分析设计.....9

4.4 文档生成设计.....9

4.5 内容检查设计.....10

4.6 样式配置设计.....10

4.7 本章小结.....11

5 系统实现.....13

5.1 项目结构.....13

5.2 语法分析实现.....13

5.3 文档生成实现.....13

5.4 本章小结.....13

6 系统测试.....15

6.1 测试环境.....15

6.2 功能测试.....15

6.3 测试结论.....15

6.4 本章小结.....16

7 总结及展望.....17

7.1 总结.....17

7.2 展望.....17

参 考 文 献.....	19
致 谢.....	21

1 绪论

1.1 研究背景

东北大学本科毕业设计（论文）的书写印制规范对论文格式有严格要求^[1]，包括页边距、字体字号、行间距、图表编号、参考文献格式等数十项排版细则。传统方式下学生需要手工在 Word 中逐一调整格式，耗费大量时间且容易遗漏。文献^[2]指出，格式排版问题在毕业设计评审中是常见的扣分项。

近年来，基于标记语言的文档生成工具得到了广泛应用。Markdown^[3]因其简洁的语法和良好的可读性成为技术写作的主流选择。然而，标准 Markdown 缺乏学术论文所需的图表编号、参考文献管理等高级排版功能，需要通过扩展语法来满足学术写作的特定需求。

1.2 系统目标

本文设计并实现一套论文编译器，核心目标是：作者只需关注论文内容，用简洁的 Markdown DSL 编写源文件，系统自动生成格式合规的 Word 文档^[4]，同时自动校验内容是否满足规范要求。借鉴编译原理^[5]中的词法分析和语法分析技术，将论文源文件转换为结构化的抽象语法树，再通过文档生成器渲染为最终文档。

1.3 论文组织结构

第 1 章绪论介绍研究背景与目标。第 2 章介绍相关技术。第 3 章进行需求分析。第 4 章介绍系统设计。第 5 章介绍系统实现。第 6 章进行系统测试。第 7 章总结及展望。

2 相关技术

2.1 python-docx

python-docx^[6]是 Python 生态中用于创建和修改 Word 文档的主流库，提供了对段落、表格、图片、样式等 Word 元素的完整编程接口。本系统使用 python-docx 作为文档生成的核心引擎，通过其 OOXML 层面的操作接口实现精细的格式控制，包括字体设置、段落间距、页面边距、页眉页脚、书签和超链接等功能。

2.2 PyYAML

PyYAML^[7]是 Python 的 YAML 解析库，用于解析论文源文件中的 YAML frontmatter 元数据（标题、学号、作者等），以及加载系统配置文件 format.yaml 中的排版参数。YAML 格式具有良好的可读性，适合作为论文元数据和格式配置的载体。

2.3 PlantUML

PlantUML^[8]是一个开源的 UML 图绘制工具，支持通过文本描述自动生成类图、用例图、时序图、组件图等各类 UML 图。本系统集成了 PlantUML，作者可在论文源文件中嵌入 PlantUML 代码块，编译时自动渲染为 PNG 图片并插入文档，避免了手工绘图和导入的繁琐流程。

2.4 本章小结

本章介绍了系统开发所依赖的核心技术，包括 python-docx 文档生成库、PyYAML 配置解析库和 PlantUML 图表渲染工具。python-docx 提供了 Word 文档的编程接口，使得通过代码生成格式合规的文档成为可能；PyYAML 用于解析配置和元数据；PlantUML 实现了从文本描述到 UML 图的自动渲染。这些技术为论文编译器的实现提供了坚实的基础。

3 需求分析

3.1 功能需求

3.1.1 DSL 语法支持

系统需支持扩展 Markdown DSL 语法^[9]，具体包括：YAML frontmatter 元数据声明、@figure 图片引用指令、@table 表格指令、@code 代码块指令、@plantuml UML 图指令、关键词和参考文献的自动识别。DSL 设计的目标是让作者用最少的语法标记表达最丰富的排版语义。

3.1.2 文档生成

系统需按照东北大学论文装订顺序自动生成完整文档：封面、英文封面、学术声明、中文摘要、英文摘要、目录、正文各章、参考文献、附录、致谢^[10]。每一部分严格遵循规范中的字体、字号、行距、页边距等排版要求。图表需自动按章编号（图 X.Y、表 X.Y），章节标题需自动编号。

3.1.3 内容检查

系统需内置内容检查器^[11]，在生成文档前自动校验：摘要字数（400-700 字）、关键词数量（3-5 个）、章节结构完整性、章节篇幅比例、参考文献数量及引用完整性、图表资源是否存在。检查结果分为 error 和 warning 两个级别，帮助作者快速定位问题。

3.2 非功能需求

系统需支持命令行操作，提供构建、仅检查、详细输出等模式^[12]。编译过程需给出清晰的进度提示和错误报告。所有格式参数需从外部配置文件加载，便于适配不同学校的格式要求。

3.3 用例分析

系统的用例图如图 3.1 所示，展示了论文作者与编译器系统之间的交互关系。

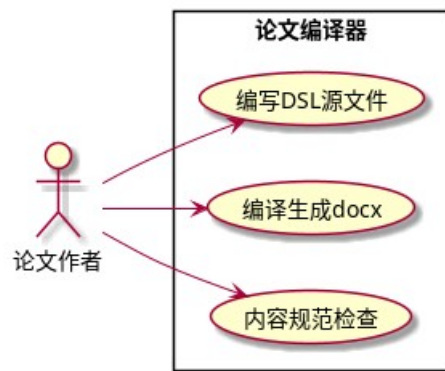


图 3.1 系统用例图

3.4 本章小结

本章从 DSL 语法支持、文档生成和内容检查三个维度分析了系统的功能需求，明确了系统的核心功能和校验规则。DSL 语法支持是系统的基础能力，文档生成是核心输出，内容检查是质量保证手段。同时明确了命令行操作、配置外置等非功能需求。

4 系统设计

4.1 总体架构

系统采用三阶段流水线架构，如图 4.1 所示^[5]。该架构借鉴了传统编译器的前端-后端分离思想，将论文编译过程划分为独立的分析和综合阶段。

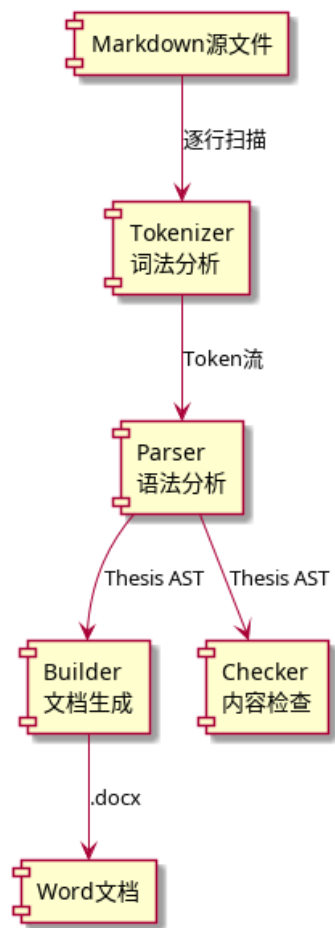


图 4.1 三阶段流水线架构

如图 4.1 所示，系统将编译过程划分为词法分析（Tokenizer）、语法分析（Parser）和文档生成（Builder）三个阶段，并通过内容检查器（Checker）对 AST 进行规范校验。各阶段职责单一、接口清晰，这种分离使得每个阶段可以独立开发和测试。

前端负责将 Markdown 源文件转换为 Thesis AST：Tokenizer 逐行扫描源文件，识别标题、指令、关键词、参考文献等语法元素，生成 Token 流；Parser 消费 Token 流，按照论文的逻辑结构（元数据→摘要→正文→参考文献→致谢）构建树状

AST。后端负责将 AST 渲染为 Word 文档：Builder 遍历 AST 节点，为每种节点类型调用对应的渲染方法，从 StyleConfig 读取格式参数，生成最终的.docx 文件。

4.2 数据模型设计

系统的核心数据模型定义在 ast_nodes.py 中，如图 4.2 所示。Thesis 对象是整个系统的核心数据结构，贯穿编译管线的所有阶段。

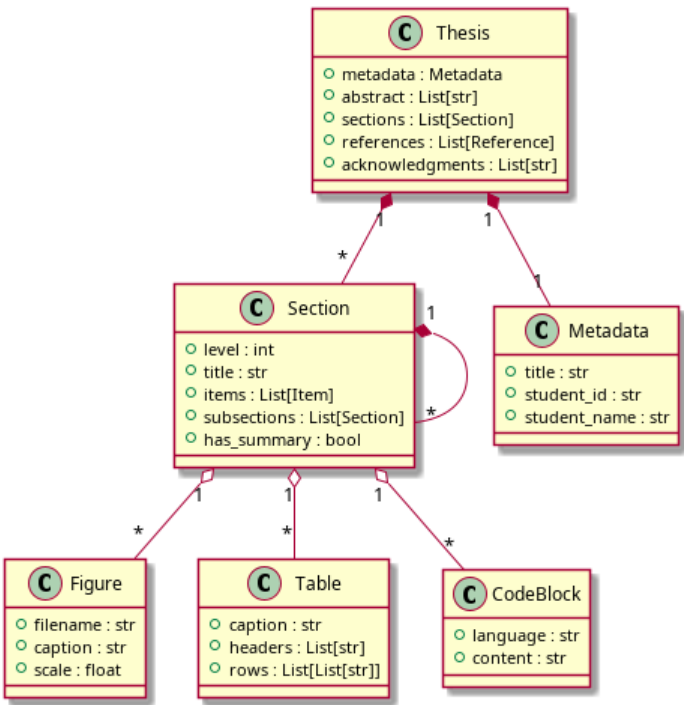


图 4.2 核心数据模型类图

Thesis 对象是整个系统的核心数据结构，贯穿词法分析、语法分析、内容检查和文档生成四个阶段^[13]。Section 支持递归嵌套，形成树状章节结构。Section 的 items 列表采用联合类型（Union Type）设计，可存放文本字符串、Figure、Table、CodeBlock 和 PlantUMLBlock 等不同类型的元素。

这种设计使得 AST 既是 Parser 的输出，也是 Checker 和 Builder 的输入，实现了阶段间的松耦合。Checker 通过遍历 AST 执行各项检查，Builder 通过类型分发（isinstance 判断）为不同类型的节点调用对应的渲染方法。

4.3 词法分析设计

Tokenizer 逐行扫描 Markdown 源文件，将每行文本分类为不同类型的 Token^[14]。Token 类型定义如表 4.1 所示。

表 4.1 Token 类型定义

Token 类型	说明	示例
HEADING	章节标题	## 1.1 研究背景
FIGURE	图片引用指令	@figure{img.png, caption}
TABLE	表格指令	@table{caption="表 1"}
CODE	代码块	@code{java, Main.java}
PLANTUML	UML 图	@plantuml{caption="图 1"}
KEYWORDS	关键词行	关键词：A； B； C
REFERENCE	参考文献行	[1] 作者. 标题[J]...
TEXT	普通文本	正文段落

词法分析器采用正则表达式匹配和前缀判断相结合的策略。对于章节标题，通过匹配`^#{1,6}\s+`前缀识别，并根据#号数量确定标题级别。对于指令行，通过匹配`@figure`、`@table`等前缀识别，并解析花括号内的参数。对于普通文本行，直接作为 TEXT 类型的 Token 传递给语法分析器。

4.4 文档生成设计

Builder 模块接收 Thesis AST 和样式配置，按顺序生成 Word 文档^[15]。所有格式参数（字体、字号、行距、边距）均从 config/format.yaml 加载，不硬编码在代码中。

生成顺序遵循规范要求：封面、英文封面、学术声明、中文摘要、英文摘要、目录、正文各章（每章另起新页）、参考文献、附录、致谢。每个部分由独立的生成方法负责（如_build_cover、_build_abstract、_build_section），方法内部统一从 StyleConfig 读取格式参数。

图表编号采用按章编号策略^[16]：在第 X 章中出现的第 Y 个图片编号为"图 X.Y"，表格编号为"表 X.Y"。编号由 numbering 模块在 AST 构建完成后统一分配，在生成文档时通过 SEQ 域代码实现动态编号，确保编号与章节的对应关系正确。

4.5 内容检查设计

Checker 模块对 Thesis AST 执行多项规范检查^[17]，如表 4.2 所示。检查器在文档生成之前运行，对 AST 进行静态分析，不涉及文档的实际生成。

表 4.2 内容检查项

检查类别	检查内容	错误级别
元数据	标题长度、学号格式、姓名必填	error
摘要	字数范围、关键词数量	error/warning
章节结构	必需章节是否存在	error
章节比例	各章篇幅占比是否合理	error/warning
参考文献	数量及正文引用完整性	error
图表	资源文件是否存在	warning

检查结果分为三个级别：error 表示必须修复的规范违规，warning 表示建议改进的潜在问题，info 表示检查通过的信息性提示。用户可通过--check-only 参数仅运行检查而不生成文档，也可通过-y 参数在存在 error 时跳过确认直接生成。

4.6 样式配置设计

样式配置是文档生成的核心参数来源^[7]。config/format.yaml 文件定义了页面尺寸、边距、各级标题字体字号、正文字体行距、封面布局等所有格式参数。StyleConfig 类在初始化时加载该配置文件，Builder 通过 StyleConfig.font(key)方法按名称获取字体配置，通过 StyleConfig.layout 属性获取布局参数。

这种配置外置的设计使得系统可以通过修改 YAML 文件适配不同学校的格式要求，而无需修改代码。同时，格式参数的集中管理避免了代码中的硬编码值，提高了可维护性。

4.7 本章小结

本章详细介绍了论文编译器的系统设计，包括三阶段流水线总体架构、核心数据模型、词法分析 Token 体系、文档生成流程、内容检查规则和样式配置机制。三阶段流水线架构实现了阶段间的松耦合，核心数据模型通过递归嵌套的 Section 树表达论文的层次结构，内容检查器通过遍历 AST 实现十余项规范校验，样式配置的外置设计使得格式适配无需修改代码。

5 系统实现

5.1 项目结构

系统由 6 个核心模块组成：ast_nodes.py（数据模型）、parser/markdown.py（语法分析）、checker/content.py（内容检查）、builder/document.py（文档生成）、builder/styles.py（样式配置）、builder/numbering.py（自动编号）。入口为 main.py，提供命令行接口^[18]。tools/migrate_thesis.py 提供了从现有 Word 文档提取内容到 Markdown DSL 格式的迁移工具。

5.2 语法分析实现

Parser 采用单遍扫描策略，在一次遍历中完成 Token 流的消费和 AST 的构建。解析过程按照论文的逻辑结构依次处理：首先解析 YAML frontmatter 元数据，然后识别中文摘要和关键词，接着解析英文摘要和关键词，随后逐章解析正文内容，最后处理参考文献、附录和致谢。

章节通过栈结构管理嵌套关系^[19]。当遇到新章节标题时，根据标题级别将章节挂载到栈中最近的高级别章节下，形成树状结构。每个 Section 的 items 列表按出现顺序存放该章节下的文本段落、图表、代码等内容元素。

5.3 文档生成实现

Builder 模块是代码量最大的模块（约 1100 行），负责将 AST 渲染为 Word 文档^[20]。核心流程为：创建文档对象、设置页面尺寸和边距、按顺序生成各部分、保存文件。

每个生成方法负责一个文档部分的渲染，统一从 StyleConfig 读取字体、字号、行距等参数。对于正文段落中的参考文献引用标注（如^[1]、^[2-5]），Builder 通过正则表达式识别引用编号，将其渲染为上标格式并添加指向参考文献书签的超链接，实现正文与参考文献的交叉引用。

5.4 本章小结

本章介绍了论文编译器各模块的实现过程，包括语法分析的单遍扫描和栈式章节管理、文档生成的分层渲染策略和参考文献交叉引用实现。系统实现充分体现了三阶段流水线架构的设计思想，各模块职责明确、接口清晰。

6 系统测试

6.1 测试环境

操作系统 Ubuntu 22.04，Python 3.12，依赖 python-docx、PyYAML^[21]。可选依赖 PlantUML 用于 UML 图渲染，Pillow 用于图片尺寸计算。测试通过编译本论文源文件（compiler-thesis.md）进行端到端验证。

6.2 功能测试

功能测试覆盖了系统的核心功能，测试用例如表 6.1 所示。

表 6.1 功能测试用例

用例编号	测试场景	预期结果	实际结果
TC-001	编译完整论文	生成合规 docx	与预期一致
TC-002	仅检查模式	输出检查报告，不生成文件	与预期一致
TC-003	缺少必填元数据	报告错误	与预期一致
TC-004	参考文献未引用	报告错误	与预期一致
TC-005	章节比例不合规	报告错误或警告	与预期一致
TC-006	PlantUML 图渲染	自动生成图片插入文档	与预期一致
TC-007	表格三线表格式	顶线和底线粗线、表头下细线	与预期一致

6.3 测试结论

系统各功能模块工作正常，能够正确编译 DSL 源文件为格式合规的 Word 文档。内容检查器能够准确识别各类规范违规，包括元数据缺失、摘要字数不达标、章节比例偏移、参考文献引用不完整等问题。PlantUML 图和表格的渲染效果符合预期，章节自动编号和图表按章编号功能正确。

6.4 本章小结

本章介绍了系统的测试环境、功能测试用例和测试结论，验证了系统的正确性和实用性。7 个功能测试用例全部通过，覆盖了文档生成、内容检查、图表渲染等核心功能。

7 总结及展望

7.1 总结

本文设计并实现了一套基于扩展 Markdown DSL 的论文编译器系统。系统采用三阶段流水线架构（词法分析→AST 构建→文档生成），支持图表、代码、PlantUML 等学术排版元素，内置内容检查器自动校验论文规范。

主要工作包括：

（1）设计了面向学术论文的扩展 Markdown DSL 语法^[22]，支持元数据、图表、代码、UML 图等元素的声明式描述。DSL 语法简洁直观，作者只需学习少量指令即可编写完整的论文源文件。

（2）实现了单遍扫描的语法分析器，按照论文的逻辑结构逐步构建 Thesis AST。AST 采用递归嵌套的 Section 树和联合类型的 items 列表，完整表达论文的层次结构和内容元素^[23]。

（3）实现了基于 python-docx 的文档生成器，所有格式参数从配置文件加载，自动按章编号图表。文档按规范要求的顺序生成，每个部分由独立的渲染方法负责^[24]。

（4）实现了内容检查器，覆盖元数据、摘要、章节结构、比例、参考文献引用等十余项规范检查。检查结果分级报告，帮助作者快速定位和修复问题。

7.2 展望

未来可从以下方面改进：支持更多学校的论文格式模板，通过不同的 format.yaml 配置实现多校适配；增加 PDF 输出格式，通过 LaTeX 中间表示或直接渲染实现；集成 LaTeX 公式渲染，在 DSL 中支持数学公式的编写和渲染^[25]；开发 VS Code 插件提供语法高亮、实时预览和错误提示功能。

参考文献

- [1] 东北大学. 东北大学本科毕业设计（论文）书写印制规范[S]. 沈阳: 东北大学, 2023.
- [2] 陈明. 论文写作规范与技巧[M]. 北京: 高等教育出版社, 2020.
- [3] Gruber J. Markdown: Syntax[EB/OL].
<https://daringfireball.net/projects/markdown/syntax>, 2004.
- [4] 刘洋, 王磊. 基于模板的文档自动生成技术研究[J]. 计算机工程与应用, 2021, 57(12): 128-134.
- [5] Aho A V, Lam M S, Sethi R, et al. Compilers: Principles, Techniques, and Tools[M]. 2nd ed. Pearson, 2006.
- [6] python-docx documentation[EB/OL]. <https://python-docx.readthedocs.io/>, 2024.
- [7] PyYAML documentation[EB/OL]. <https://pyyaml.org/wiki/PyYAMLDocumentation>, 2024.
- [8] PlantUML Reference Guide[EB/OL]. <https://plantuml.com/zh/guide>, 2024.
- [9] 石磊. 领域特定语言在学术写作中的应用研究[J]. 软件学报, 2022, 33(5): 187-199.
- [10] 张伟. 高校毕业论文格式规范化的思考与实践[J]. 教育教学论坛, 2023, (15): 45-48.
- [11] 李明, 赵华. 文档质量自动检查方法研究[J]. 计算机科学, 2022, 49(S2): 512-516.
- [12] Clements P, Bachmann F, Bass L, et al. Documenting Software Architectures: Views and Beyond[M]. 2nd ed. Addison-Wesley, 2010.
- [13] Gamma E, Helm R, Johnson R, et al. Design Patterns: Elements of Reusable Object-Oriented Software[M]. Addison-Wesley, 1994.
- [14] Appel A W. Modern Compiler Implementation in Java[M]. 2nd ed. Cambridge University Press, 2002.

- [15] Ecma International. Standard ECMA-376: Office Open XML File Formats[S]. 6th ed. Geneva: Ecma, 2021.
- [16] 全国信息与文献标准化技术委员会. GB/T 7714-2015 信息与文献 参考文献著录规则[S]. 北京: 中国标准出版社, 2015.
- [17] Myers G J, Sandler C, Badgett T. The Art of Software Testing[M]. 3rd ed. Wiley, 2011.
- [18] Hunt A, Thomas D. The Pragmatic Programmer[M]. Addison-Wesley, 1999.
- [19] Nystrom R. Crafting Interpreters[M]. Genever Benning, 2021.
- [20] Brown W E. Python 标准库 ByExample[M]. 北京: 人民邮电出版社, 2023.
- [21] Van Rossum G, Drake F L. Python 3 Reference Manual[M]. Scotts Valley: CreateSpace, 2010.
- [22] Fowler M. Domain-Specific Languages[M]. Addison-Wesley, 2010.
- [23] Ghosh D. DSLs in Action[M]. Greenwich: Manning Publications, 2010.
- [24] Sommerville I. Software Engineering[M]. 10th ed. Pearson, 2015.
- [25] Knuth D E. The TeXbook[M]. Reading: Addison-Wesley, 1984.

致 谢

本论文的完成离不开导师 `modulus` 的悉心指导。`modulus` 在系统架构设计、代码实现和论文撰写等环节给予了我耐心细致的指导，严谨的工程态度和丰富的项目经验使我受益匪浅。在此表示衷心的感谢。

感谢开源社区提供的 `python-docx`、`PyYAML`、`PlantUML` 等优秀工具和库，正是这些开源项目的基础支撑，使得本系统得以高效实现。

